RUNBOOK

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

SDDC Manager Resource-Lock & Post-Redeploy DB Cleanup

Clearing stale deployment locks (and related post-redeploy state) that block SDDC Manager operations.

SDDC MANAGER | RESOURCE LOCKS | PLATFORM DB | TRUST AUTH | PG_HBA | POST-REDEPLOY | RUNBOOK

v1.2 VMware Cloud Foundation 9

SDDC Manager Resource-Lock & Post-Redeploy DB Cleanup

Prepared by: Virtual Control LLC **Date:** June 9, 2026 **Document Version:** 1.2 **Classification:** Internal

When to use. A SDDC Manager operation (e.g., *Configure Backup*) fails at **"Acquire Lock"** with "*Deployment level lock cannot be acquired. There are existing resource locks.*" The locks are stale leftovers from failed/post-redeploy operations. **The public API cannot force-release them** — clearing is a direct **platform-DB** edit. This is the established Virtual Control procedure (used repeatedly after the vCenter redeploy).

Table of Contents

SDDC Manager Resource-Lock & Post-Redeploy DB Cleanup

1. Symptom & Diagnosis
2. Pre-Flight (read-only, mandatory)
3. The Fix — Platform-DB Cleanup (verified 2026-06-09)
   - 3.1 Access
   - 3.2 Complete verified sequence (run as root, in order)
   - 3.3 Why all four statements, not just DELETE FROM lock
   - 3.4 Entering the commands — restricted-shell paste caveat (the EOF trap)
   - 3.5 Checkpoint — confirm trust is actually applied before the fixes (2026-07-09)
4. Verify
5. This Instance — Record
6. Document Information

   Revision History

---

1. Symptom & Diagnosis

- A workflow fails at **"Acquire Lock on SDDC Manager"** → "*Deployment level lock cannot be acquired. There are existing resource locks.*"
- `GET /v1/resource-locks` shows locks held, but **no task is In Progress** → the locks are **stale**.
- **The API does not support releasing locks.** `DELETE /v1/resource-locks/{id}` returns **HTTP 400 /** `ANNOTATIONS_MISMATCH "Request method 'DELETE' is not supported"` (verified 2026-06-09). Per the SDDC Manager API

> Handbook §7: "*The API provides no way to force-release locks … direct database cleanup (`DELETE FROM Lock`) is required.*"

This instance (2026-06-09): 4 stale locks blocking the SDDC Manager backup config — `domain/mgmt`, `system/SYSTEM`, `vcenter/vcenter`, `nsx/nsx-vip.lab.local` — all held by `serviceId = OPERATIONS_MANAGER`, no in-progress task.

---

2. Pre-Flight (read-only, mandatory)

```
 # Token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<sso-pw>"}' | python3 -c "import
sys,json;print(json.load(sys.stdin)['accessToken'])")

# 1) List the locks
curl -sk -H "Authorization: Bearer $TOKEN" https://sddc-manager.lab.local/v1/resource-locks | python3 -m
json.tool

# 2) GATE: confirm ZERO tasks are genuinely running — do NOT proceed if any are In Progress
curl -sk -H "Authorization: Bearer $TOKEN" "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS"
```

> **Only proceed if `IN_PROGRESS` is empty.** Deleting a lock held by a live operation can corrupt that operation. The locks must be confirmed **stale** (held with no running task).

---

3. The Fix — Platform-DB Cleanup (verified 2026-06-09)

> **The PostgreSQL password is not discoverable** in the VCF configuration. The only supported way in is the **trust-auth workaround**: temporarily set `trust` in `pg_hba.conf`, run the fixes over the local `127.0.0.1` connection, then **restore `pg_hba.conf` immediately**. Verified paths on SDDC Manager 9.0.1: data directory `/data/pgdata`, control binary `/usr/pgsql/15/bin/pg_ctl`.

3.1 Access

SSH is permitted **only** as `vcf` (root and admin SSH are rejected by the appliance); escalate with `su -`:

```
 ssh vcf@sddc-manager.lab.local # vcf credential — see vault
su - # root credential — see vault
```

3.2 Complete verified sequence (run as root, in order)

Enter these as **plain one-line commands** — *not* a heredoc (see §3.4). The block backs up `pg_hba.conf`, enables trust, reloads, runs all four fixes in one session, then **unconditionally restores** `pg_hba.conf` and restarts the scheduler:

```
 # 1) Back up pg_hba.conf (CRITICAL — this is the revert point)
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak

# 2) Enable trust (covers scram-sha-256 and md5) + reload (no restart needed)
sed -i -E 's/(scram-sha-256|md5)/trust/g' /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# 3) The four fixes — all in one session (the prevalidation reads all of them)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt SET status = 'ACTIVE' WHERE
status != 'ACTIVE';\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE task_metadata SET resolved = true
WHERE resolved = false;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM task_lock;\""

# 4) Confirm the lock table is empty
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT count(*) AS remaining_locks FROM
lock;\""

# 5) RESTORE pg_hba.conf + reload — do NOT skip; never leave trust enabled
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# 6) Restart the orchestration service so it re-reads clean state
systemctl restart operationsmanager
```

3.3 Why all four statements, not just `DELETE FROM lock`

The `nsxt`, `lock`, and `task_metadata` tables all feed the **same deployment prevalidation check** — clearing only the lock table is not enough when the appliance also carries post-redeploy state. Running the four together in one session also clears the **NSX/vCenter** `ACTIVATING` and **stuck-task** conditions in the same pass.

> **Restore is mandatory.** `trust` makes the local PostgreSQL passwordless — a real security exposure. Step 5 copies the pristine `.bak` back and reloads, returning every `host` rule to `scram-sha-256`. Confirm with §4 before walking away. This is platform-DB surgery and is the established Virtual Control procedure for the post-redeploy state — **not** a Broadcom-published self-service step (the Broadcom KB for this error routes to Support). Run only with appliance root access, in a known-quiet window. *(Lab policy: no SDDC Manager snapshot — rely on the file-based backup and the verify step.)*

3.4 Entering the commands — restricted-shell paste caveat (the `EOF` trap)

> **Do not paste a `cat > /tmp/file <<'EOF' … EOF` heredoc into the SDDC Manager shell.** The appliance terminal **auto-indents pasted lines**, which pushes the closing `EOF` off column 0. A quoted heredoc only

terminates when `EOF` is the **first character on the line with no leading whitespace** — so the shell never closes the document. It sits at the `>` continuation prompt and silently swallows the trailing `bash …` line into the file.

Symptoms observed 2026-06-09: repeated `>` prompts; every body line arriving indented as `> #!/bin/bash`; the script written but never executed; the file ending up with a literal `EOF` line and a `bash /tmp/fix_locks.sh …` line as **content** (which, if run, would self-recurse and `rm` itself).

Reliable entry methods (in order of preference):

- **Plain one-line commands** (as in §3.2) — leading whitespace is harmless for ordinary commands; only a heredoc *terminator* cares about column 0. **This is the method that worked.**
- **Base64 one-liner** — `echo '<base64>' | base64 -d > /tmp/fix.sh && bash /tmp/fix.sh; rm -f /tmp/fix.sh` — one unbroken line the auto-indent cannot break. Generate locally with `tr -d '\r' < script.sh | base64 -w0`.
- If a heredoc is unavoidable and you are stuck at `>`: press **Ctrl-C**, `rm -f` the partial file, and switch to one of the above. Do **not** keep typing — the file is already malformed.

File transfer note. `scp` does not work to SDDC Manager (restricted shell). Push a file with `ssh vcf@host "tr -d '\r' > /home/vcf/file" < localfile` (the `tr -d '\r'` strips Windows CR so bash does not choke on `\r`).

3.5 Checkpoint — confirm `trust` is actually applied before the fixes (2026-07-09)

If a `psql` fix returns `psql: error: … password authentication failed for user "postgres"`, `trust` was never applied — almost always because the `sed` (enable-trust) line was **skipped on paste** or did not match the rule. The four fixes then fail: they still need a password, which is **not discoverable**. Do **not** try to supply a postgres password — there isn't one.

Always verify `trust` is live *before* running the fixes:

```
grep -E 'trust|scram-sha-256|md5' /data/pgdata/pg_hba.conf | grep -v '^#'
```

Every `host … 127.0.0.1/32` and `::1/128` line must read **`trust`** — not `scram-sha-256`. If they still read `scram-sha-256`, re-run the `sed` + `pg_ctl reload`, re-check with this grep, and only then run the four fixes. *(2026-07-09: the `sed` was skipped on the first paste → `DELETE FROM Lock` hit "password authentication failed for user postgres"; re-running `sed` + reload + this grep-verify resolved it and the fixes ran clean.)*

---

4. Verify

Database-level (authoritative):

```
 # Lock table empty (run during the trust window, or after restore via API)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT count(*) AS remaining_locks FROM lock;\""
```

Auth fully reverted (do this every time):

```
 # Every host rule must read scram-sha-256; trust/md5 should appear ONLY in comment text
grep -nE 'scram-sha-256|md5|trust' /data/pgdata/pg_hba.conf | grep -vE '^[0-9]+:[[:space:]]*#'
systemctl is-active operationsmanager
```

- `remaining_locks` → **0**.
- Every uncommented `host … 127.0.0.1/32` / `::1/128` rule reads **`scram-sha-256`**. A nonzero `grep -c trust` can be misleading — the stock `pg_hba.conf` header mentions "trust" in **comments**; confirm by reading the actual rule lines, not the count.
- `operationsmanager` → **`active`**.
- **API cross-check (optional):** `GET /v1/resource-locks` → 0; `GET /v1/nsxt-clusters` / `GET /v1/vcenters` — `ACTIVATING` should clear toward `ACTIVE`.
- **Retry the blocked operation** (e.g., *Configure Backup → Backup Now*); it should now acquire its lock.

---

5. This Instance — Record

Field	Value
Date	2026-06-09
Trigger	*Configure Backup of VCF Components* failed at "Acquire Lock"
Locks found	4 — `domain/mgmt`, `system/SYSTEM`, `vcenter/vcenter`, `nsx/nsx-vip.lab.local` (all `OPERATIONS_MANAGER`)
In-progress tasks at gate	0 (safe to clear)
API delete attempt	HTTP 400 — "Request method 'DELETE' is not supported" (confirms DB path)
Access discovered	SSH `vcf` only → `su -`; psql password **not discoverable** → trust-auth workaround required. Socket exists at `/home/postgresql` but no `pg_hba` rule for `[local]`; `-h 127.0.0.1` rule was `scram-sha-256` (password) → flipped to `trust` to get in.
Paths confirmed	`pg_hba.conf` = `/data/pgdata/pg_hba.conf`; control = `/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata`; PG 15
Entry-method snag	Heredoc paste auto-indented → `EOF` never closed (§3.4). Resolved by using plain one-line commands.
Results	`UPDATE nsxt` → **1** · `DELETE FROM lock` → **4** · `UPDATE task_metadata` → **164** · `DELETE FROM task_lock` → **0** · `remaining_locks` → **0**

Field	Value
Restore verified	pg_hba `host` rules back to `scram-sha-256`; `operationsmanager` → `active`; backup left at `/data/pgdata/pg_hba.conf.bak`

Second instance — 2026-07-09. The scheduler jam from 2026-07-06 (349 tasks `IN_PROGRESS`, a stuck credential-rotate holding the locks) had **self-cleared** by 07-09 — the stuck tasks aged out to Failed/Cancelled, leaving **0** `IN_PROGRESS` (gate satisfied) and the same **4 stale locks** (`domain`, `system`, `vcenter`, `nsx`). Ran the procedure; the enable-`trust` `sed` was skipped on the first paste → `psql: password authentication failed for user postgres` → re-ran `sed` + reload, verified `trust` via grep (§3.5), then the four fixes ran clean — `UPDATE nsxt` **1**, `DELETE lock` **4**, `UPDATE task_metadata` **175**, `DELETE task_lock` **0**, `remaining_locks` **0**; pg_hba restored to `scram-sha-256`, `operationsmanager` → `active`. **Verified via API: locks 4 → 0, 0** `IN_PROGRESS`**,** `operationsmanager` **reachable.** Note: immediately after the `operationsmanager` restart the lock list read transiently as **4** (stale during the service restart) then settled to **0** — always re-verify once the service is fully back up, not the instant it restarts.

---

6. Document Information

Field	Value
Title	SDDC Manager Resource-Lock & Post-Redeploy DB Cleanup
Document ID	VC-RUNBOOK-SDDC-LOCK-DBCLEAN-20260609
Version	1.2
Issued	2026-06-09
Author	Virtual Control LLC
Classification	Internal
Platform	VMware Cloud Foundation 9.0.1 (SDDC Manager 9.0.1.0, PostgreSQL 15)
Source procedures	VCF9-Master-Bible (trust-auth workaround, `/data/pgdata`); VCF-Interview-CheatSheet (DB-repair sequence); VCF-SDDC-Manager-API-Handbook §7 (Resource Locks)
Related Documents	VCF9-Outstanding-Issues-Landscape-2026-06-09; PLAN-NSX-Recovery-Options-and-Clean-Rebuild-2026-06-09
Distribution	Lab notebook

Revision History

Version	Date	Notes
1.0	2026-06-09	Captured the verified resource-lock clearing procedure after the *Configure Backup* "Acquire Lock" failure: confirmed the API cannot release locks (HTTP 400 "DELETE not supported"), documented the platform-DB `DELETE FROM lock` fix and the full post-redeploy repair sequence (UPDATE nsxt / DELETE lock / UPDATE task_metadata / DELETE task_lock / restart operationsmanager), the read-only safety gate, verification, and this instance's 4 locks.

Version	Date	Notes
1.1	2026-06-09	Rewrote §3 from the **live execution**, not recollection: added the mandatory **trust-auth workaround** (password not discoverable → back up `pg_hba.conf`, `sed scram/md5→trust`, reload, fix, **restore**, reload) with verified paths (`/data/pgdata`, `/usr/pgsql/15/bin/pg_ctl`); added §3.4 the `EOF` **heredoc paste trap** (auto-indent breaks the terminator → use plain one-liners or base64) and the `scp`-blocked file-transfer note; expanded §4 to verify auth reversion (`scram-sha-256`) and service state; recorded actual results (nsxt 1 / lock 4 / task_metadata 164 / task_lock 0 / 0 remaining) and the access path in §5.
1.2	2026-07-09	**Second run.** The 07-06 scheduler jam had **self-cleared** to 0 `IN_PROGRESS` by 07-09; cleared the same 4 stale locks (`domain` / `system` / `vcenter` / `nsx`). Added **§3.5** — verify `trust` is actually applied (grep the `127.0.0.1` rules) **before** the fixes, after the enable-`trust` `sed` was skipped on paste and `DELETE FROM lock` hit "*password authentication failed for user postgres*" (no discoverable PG password — do not try to supply one). Added the transient post-restart stale-lock-read note (read 4, settled to 0). Recorded the second instance in §5; verified via API (locks 4→0, 0 in-progress, `operationsmanager` reachable).

Document End